Homework Assignment 03

Objective:

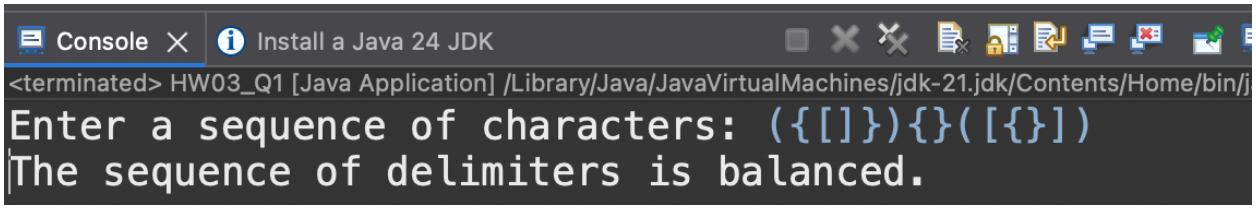
- This assignment is designed to test your understanding and ability to apply data structures such as stacks and queues in solving algorithmic problems. You will also delve into the basics of string manipulation and balanced parentheses validation using a stack.

Q01. Balanced Delimiters:

Explains your approach and methodology.

- For question one my approach was to first prompt the user for a sequence of characters in order to have a sequence to test whether the delimiters are balanced. This input was saved into a string and imported to the boolean method (isBalanced) where the stack was being used. The reason for this method to be a boolean was to incorporate it into an if statement and based on the value the program will tell the user if the sequence is balanced or not. In the isBalanced method The loop iterates through the imputed sequence and initializes it into a char for every iteration of the loop. Once the char is assigned its value, the approach I took was to have the char pushed into the stack if it was a ‘(‘, ‘{‘, or ‘{‘. If the char happened to be a ‘)’, ‘}’, or ‘}’ and if the stack happened to be empty the method would return false due to a closing delimiter being in the sequence before an opening one. Otherwise if the stack wasn’t an empty stack then it would pop a char from the stack and assign it to a character called top. The next step to the program involves using another boolean method that's only purpose was to give a true or false value based on if the char and the top of the stack (that has been popped) are a match, if they were the method would increment and do the isBalanced method again until the input sequence was fully read through. If they happened to not be a match the method would return a false and the main method would say that the sequence is not balanced.

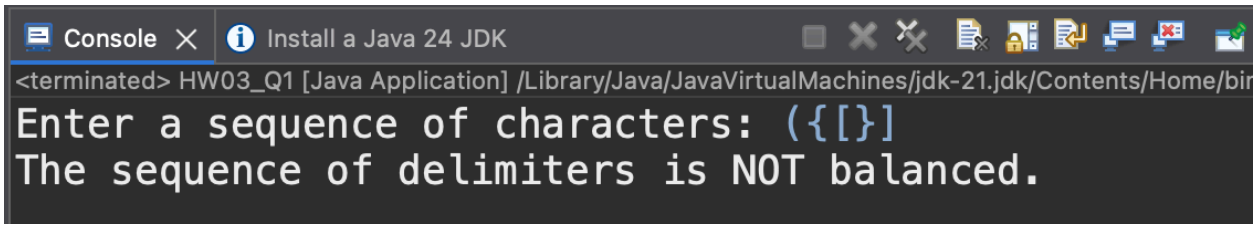
Test cases:



This test case showcases the sequence that is entered is determined to be a balanced sequence by the program.



This test case showcases the sequence that begins with a delimiter that is a closing delimiter and therefore they are not balanced due to the fact that there needs to be an opening delimiter of the same type prior to a closing one.



This test case showcases the sequence that is entered is not balanced due to there being a missing delimiter (‘]’) that would close the [in the sequence.

Discuss the time complexity of your solution.

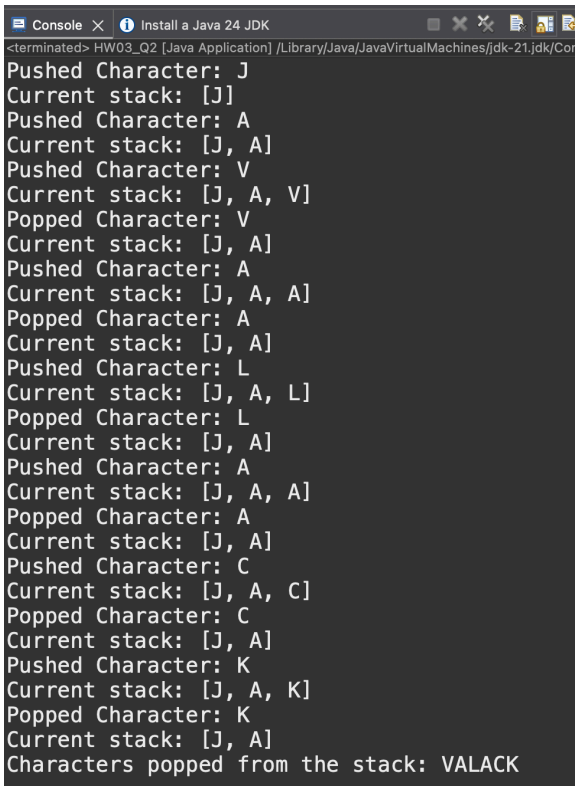
- The time complexity of this code is O(n) where n would be the length of the sequence. There is a chance for the code to be O(1) and this would only happen in the case where the first character of the sequence would be a closing delimiter. Under these circumstances the for loop would only run once and would instantly return a false value because the stack is empty to begin with therefore the first delimiter can’t be a closing one.

Q02. Stack Sequence Analysis:

Explains your approach and methodology.

- My approach for this program was to have have the given sequence (J A V # A # L # A # C # K #) hardcoded into the program and to have a string for poppedChars that is initially empty but is later appended. Once these two were initialized my next step was to create a loop that takes every char from the sequence and check weather it is a letter or #. If the current char was a not a # the program would push the current char into the stack and display the current stack to the user. If the char is a # and if the stack wasn't empty the stack would pop the current char and append it to the string of poppedChars as well as print the currently popped char. However, if the stack was empty to begin with and the first char was a # then the program would return a statement that told the user that the stack was empty. After every step the current stack was also printed to showcase the steps taken and once the end of the loop was reached the string of popped chars was printed.

Test Case:



This test case is the output of the program for when the preset sequence "JAV#A#L#A#C#K#" is hard coded as the sequence in the code that is being worked with.

Discuss the time complexity of your solution.

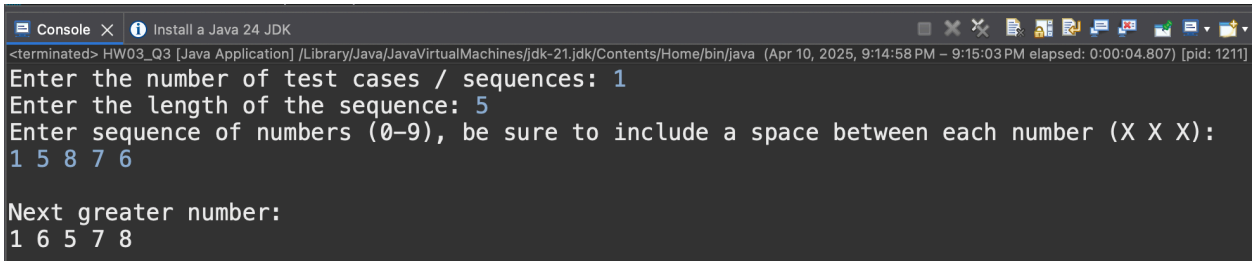
- The time complexity of this code is O(n) where n would be the length of the sequence. This is due to there being a loop in the code and for any data n.

Q03. Next Greater Number Using Stack and Queue:

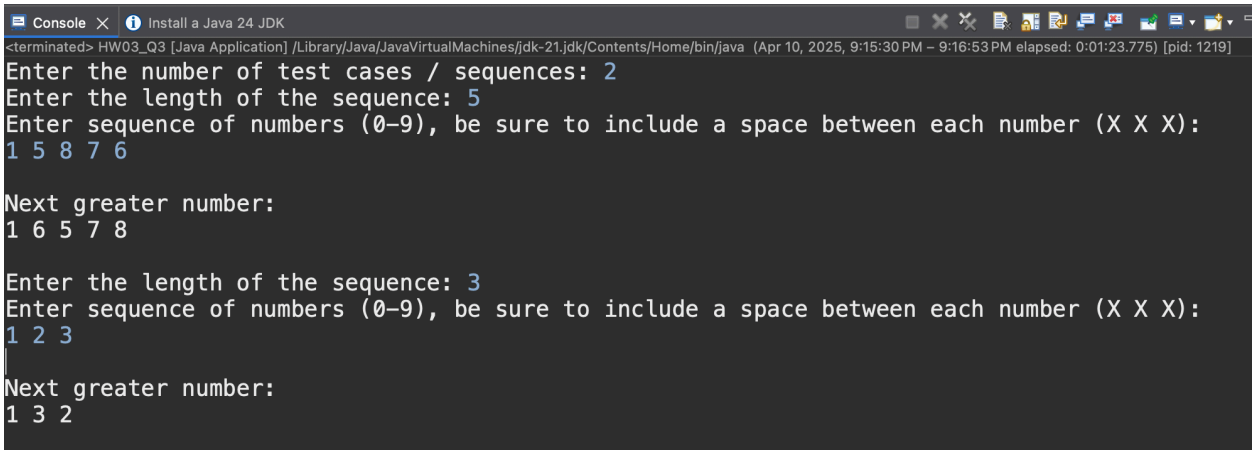
Explains your approach and methodology.

- My approach for this program was to allow the user to input multiple test cases. With the opportunity to input this, each case was allowed to have a different sequence of numbers. The user was initially prompted to see whether there will be multiple test cases or only one. Going from there if there was only one the user would have to input the length of the sequence and after hitting enter the user needs to enter the sequence of numbers with each number being between 0-9 and needing a space in between each number. In order to find the next greater number the program starts from the back of the array that has the sequence and looks for the first instance where the number behind the one being currently looked at is smaller. This number would be called the pivot, however if there is no smaller number found then the program assumes that there can be no greater number formed as the number is already at its greatest state. Now that the pivot was found the program looks for the second smallest number, or smallest excluding the pivot in order to swap the two. After the swap was completed, the program sorts the numbers after the pivot to make the smallest possible number with the remaining ones. This prints out the new number which is the next biggest number. If there are more than one test cases this code asks the users again for the length of the sequence of the new sequence of numbers as well as asking for the sequence itself,

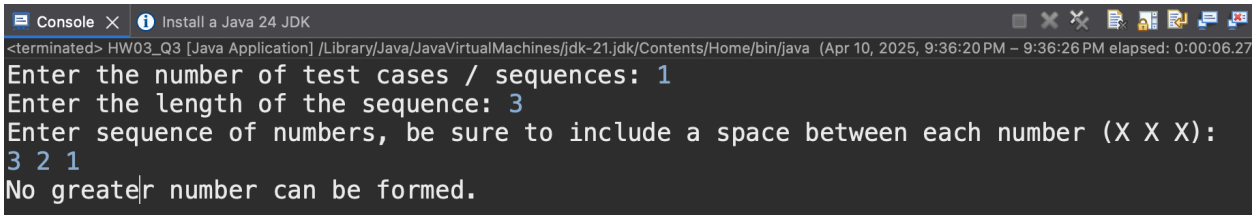
Test Cases:



This test case showcases a situation where the user wants to input one test case or sequence with the length of 5. The program finds the next greater number of 15876 to be 16578.



This test case showcases a situation where the user wants to input two test cases or sequences with the length of 5 and 3. The program finds the next greater number of the first sequence 15876 to be 16578. The program later asks for the next sequence and finds that the next greater number of sequence 123 is 132.



This test case showcases a situation where the user wants to test a sequence that is 3 2 1 and of length 3. However, due to the sequence being already in its greatest form, no greater number can be formed.

Discuss the time complexity of your solution.

- The time complexity of my solution would be O(n) x O(n log n) or O(n^2 log n). The program has one main for loop which gives the program its O(n) time complexity. The Array.sort has a time complexity of O(n log n), therefore if you were to combine the two by multiplying them together the time complexity would be O(n^2 log n). The reason for the two complexities being multiplied and not added together is due to the Array.sort being nested in the for loop.